# Follow the Drops — XRPL AML detector: method report

*Decode & Detect: The XRPL AI Hack · team Follow the Drops (captain and only member: Francesco Coccia) · 24 Oct 2026. "(our data)" marks our own measurements; every other claim links a source we opened.*

## 1. Problem

We score every XRPL payment in a one-week test window with a 0–1 laundering suspicion score. The ranking is 100 × average precision (AP) on hidden labels.

- AP sums precision at each recall step, so only the **order** of scores matters, not calibration. (scikit-learn AP)
- The intended user is a VASP compliance analyst working through an alert queue.
- Rules-based monitoring is known for very high false-positive rates, as noted by Weber et al. (arXiv 1908.02591)

## 2. Data

- **Format.** Synthetic XRPL-shaped tables: transactions, transfer legs, ledgers and hourly market rates.
  - The organiser helper `load_payments` projects each payment to one effective sender→receiver edge.
  - Values are normalised to synthetic reference USD from `delivered_amount` and the hourly rates.
  - On XRPL, `delivered_amount` rather than `Amount` is the reliable figure. (partial payments)
- **Size.** Train has 40,131 payments, of which 1,250 are positive (3.1%), from 676 senders. Test has 40,101 payments. Train and test share no wallets (our data).
- **Labels.** Each label carries a family, a difficulty and a scenario.
  - 9 typology families (fan-in/out, chain, cycle, stack, bipartite, multi-stage, random) follow the AMLworld design. (Altman et al. 2306.16424)
  - There are also two relay families and 25 "ambiguous" positives.
  - Relays are ~200 positives hidden among ~19,800 structurally identical decoys (600 scenarios each, 6 positive). Other families have planted look-alike decoys (our data).
- **Time.** Timestamps lie on a 2 s grid; the minimum gap between distinct timestamps is 4 s and the median 12 s (our data). Mainnet rounds `close_time` to 10 s. (ledger close times)
- **Not features.** Family, difficulty, scenario and cohort are never used as features.

## 3. Validation design

- **Folds.** `GroupKFold(5, shuffle=True, random_state=42)` grouped by `scenario_id`.
  - Unlabelled background rows get a unique id.
  - So every member of a scheme or decoy set lands in a single fold. (scikit-learn GroupKFold)
- **Reported metrics.** Mean ± std AP over folds, per-fold AP, precision and recall in the top 1% of each fold, AP beyond the top 1%, and per-family / per-difficulty AP.
- **Why not sender grouping.** Grouping by sender (StratifiedGroupKFold, docs) gave 69.1 vs 53.3 for v1: an optimistic estimate, because one scheme spans many senders. The wallet graph is one connected component, so component grouping is impossible (our data).
- **Leakage controls.**
  - Any score-derived quantity (smoothing, stacking) uses same-fold out-of-fold scores only.
  - The test set is never used for tuning.

## 4. Features (247 columns, `src/features.py`, `src/flows.py`)

| group | n | what it measures |
|---|---|---|
| Wallet velocity windows | part of 175 | in/out counts, sums and unique counterparties per wallet over 10 min–7 days; forward and fund ratios |
| Pair, holding and structuring | part of 175 | pair novelty and share (`pair_total` is a top feature), holding time, forwarding lag, amount similarity of the next outflow |
| Multi-hop and graph | part of 175 | 1–3-hop flow, cycle return, chain score, clustering and degree (networkx); per-episode balance conservation |
| **Flow reconstruction** | 60 | label-free "flows" (below): size, duration, rate, senders/receivers/wallets, density, value spread, relative value, position |

| group | n | what it measures |
|---|---|---|
| **Gap regularity** | 12 | log median gap between consecutive payments, and gap coefficient of variation, per flow window |

- **How flows are built.** Two payments are linked when they share a wallet and follow each other on that wallet within W seconds.
  - Undirected flows use W = 30/120/600 s.
  - "Forward" flows use W = 60/600/3600 s and link only an inflow to that wallet's next outflow.
  - A flow is a connected component of these links. (SciPy connected_components)
- **Why flows.** Test data has no scenario ids, so flows rebuild scheme-like units from the ledger alone. This follows the finding that laundering shows in patterns across transactions, not in single rows. (Altman et al.)
- **FATF red flags covered:** rapid pass-through, structuring, many new counterparties. (FATF red flags)
- **Future context.** Features use both earlier and later activity in the window, so the system is a batch / T+1h investigation tool. A streaming version needs past-only windows.

### 5. Models, blend and smoothing (`src/predict.py`)

- **Members.**
  - LightGBM (lr 0.03, 600 trees, 31 leaves, subsample 0.8, colsample 0.6), averaged over seeds 42–46.
  - sklearn HistGradientBoosting (lr 0.05, 500 iterations, 31 leaves, min_samples_leaf 40, l2 1.0). (HGB docs)
  - Logistic regression.
  - This GBT-on-graph-features family matched GNN baselines on AMLworld. (Altman et al.)
- **Training weights.** Relay and ambiguous positives get sample_weight 0, because they are indistinguishable from their decoys (our data). They are still scored.
- **Blend.** Rank-normalised average with weights LGB:HGB:LR = 2:1:1.
- **Smoothing.**
  - score = rank(s) + 0.5·rank(max s in the payment's 600 s undirected flow) + 0.5·rank(max s in its 3600 s forward flow).
  - A tiny 1e-6·rank(LGB) term breaks ties, so the CSV has no tied scores.
- **Explanations.** `src/explain.py` uses LightGBM's per-feature contributions (`pred_contrib=True`, log-odds; LightGBM predict).
  - It writes the top alerts with their reconstructed flow and three plain-English reasons to `outputs/example_alerts.md`.
  - Example, the #1 test alert: 12 payments through 8 wallets in 228 s; 6 of 8 wallets both receive and send, forwarding 96% of what they receive. Top reasons: 2-min flow size, 10-min flow density, one-off pair (our data).

### 6. Results (our data)

| version | what changed | CV AP (mean ± std) | public LB |
|---|---|---|---|
| starter | organiser baseline | 28.25 | — |
| v1 | LightGBM on 175 graph/pair/velocity features | 53.27 ± 6.97 | 57.4 |
| v2 | + 60 flow features, LGB+HGB+LR rank blend | 59.46 ± 5.82 | 63.5 |
| v3 | + 12 gap features, relay weights, 2:1:1, flow smoothing | **62.22 ± 4.83** | pending |

- **v3 per fold:** 61.5 / 67.2 / 67.1 / 61.3 / 54.0, better than v2 in 5/5 folds.
- **Review budget.** In the top 1% of each fold, precision is 0.875 and recall 0.305. Pooled, 401 alerts contain 351 laundering payments. AP beyond the top 1% is 41.1, up from 37.3 for v2.
- **Within-family AP, v2 → v3:** fan_in 89.5 → 90.2 · multi_stage 86.5 → 89.4 · fan_out_in 83.2 → 86.4 · cycle 85.5 → 86.3 · fan_out 82.5 → 84.5 · stack 80.5 → 81.6 · chain 73.5 → 79.4 · random 66.7 → 74.7 · bipartite 64.0 → 58.5 (the only loss) · relays ~1 → ~1 (chance).
- **By difficulty (v3):** easy 81.7, medium 9.8, hard 0.7.
- **False alarms are look-alikes.** Among the 250 highest-scored out-of-fold negatives, only 9 (3.6%) are background traffic; among the top 1,250, 155 (12%). The rest are planted decoys, mostly relay look-alikes.

- **Top LightGBM features by gain:** `fl120_size`, `pair_total`, `fl120_nw`, `fl120_density`, `fl600_density`, `fl600_rate`.

### 7. What failed (our data, same folds)

- **Relay "hub" features (+47 columns).** 50.8 vs 53.3. An internal audit found that a relay-vs-rest classifier reaches AUC 0.986 (audit measurement, not re-run by us), but nothing separates relay positives from their identical decoys.
- **Stage-2 stacking on neighbour scores: the leakage lesson.**
  - Cross-fold neighbour scores showed a +3.1 AP "gain". It was leakage: the neighbours' out-of-fold scores came from models that had seen this row's label.
  - Done fold-locally, stage-2 alone scored 56.7 and hurt the blend. Strict nested stage-2 in phase 2 also lost (44.3 / 52.0).
- **No gain (inside noise):** chain-depth features (59.47 vs 59.46), value-gated long flows (AUC 0.60–0.65 alone), v1 as a fourth member, extra windows, LGB tuning without flows.
- **Medium tier.** Medium scenarios last about 1–8 h, with median gaps of 1,660 s vs 2,108 s for normal wallets. They are not separable with our features.

### 8. Limitations

- Relays and ambiguous labels are not separable from ledger data alone.
- Medium and hard tiers are weak.
- High fold variance (±4.8), from one week and 676 senders.
- Batch (uses future context), not real time.
- The data is synthetic, so performance may not transfer one-to-one to real chains. (Altman et al., App. B)
- Models drift after market events, as seen on Elliptic. (Weber et al.)
- No off-chain KYC.
- Outputs are review prompts for a human analyst, never allegations or automatic actions.

### 9. Reproducibility

- **Environment.** Python 3.13.5; pinned `requirements.txt` (pandas 3.0.6, numpy 2.5.4, scikit-learn 1.9.1, lightgbm 4.7.0, scipy 1.18.1, networkx 3.7, pyarrow 26.0.0, tabulate 0.10.0).
- **Seeds.** All 42; LightGBM final members use seeds 42–46.
- **Commands:**

```
pip install -r requirements.txt
# CV report -> outputs/cv_results.md
python src/train.py   --train-dir TRAIN --cache-dir /tmp/xrpl-cache
# final predictions -> outputs/predictions.csv (= predictions_v3.csv)
python src/predict.py --train-dir TRAIN --test-dir TEST --cache-dir /tmp/xrpl-cache
# worked alerts -> outputs/example_alerts.md
python src/explain.py --train-dir TRAIN --test-dir TEST --cache-dir /tmp/xrpl-cache
```

- **Runtime.** Cold, on 8 cores: train.py ~186 s, predict.py ~63 s, about 4.2 min in total. explain.py takes ~5 s with a warm cache.
- **Output checks.** `predict.py` checks one row per test transaction, scores in [0, 1], no ties and size under 12 MiB.
- **Hygiene.** No data in the repo.
- **Licence.** MIT; third-party components and AI assistance are listed in `ATTRIBUTION.md`.

### 10. Team

**Follow the Drops** — Francesco Coccia (captain, sole member).